JAVA 소개

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Introduction to
Java
C++

JAVA란?

컴퓨터 프로그래밍

컴퓨터에서 동작하는 프로그램을 만드는 것을 의미

문제를 해결하기 위해 방법과 절차를 프로그래밍 언어로 정의하는 과정

알고리즘

- 문제해결을 위해 실행하는 절차의 목록
- 특정 문제를 해결하기 위한 단계별 행동을 정의한 것

예 요리 레시피나 게임 미션 해결하기, 종이 접기 등

JAVA란?

프로그래밍 언어

프로그래밍 언어의 역할은 컴퓨터가 연산처리를 하도록 명령하는 것

프로그램은 컴퓨터의 CPU가 읽을 수 있는 기계어로 번역되어 실행

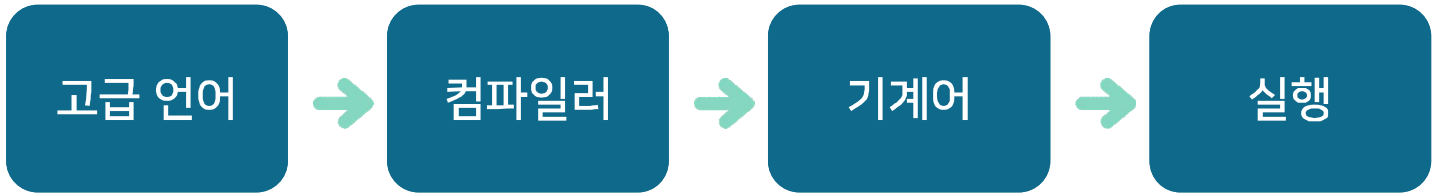
프로그래밍 언어는 고급 언어와 저급 언어로 구분

- 고급 언어는 사람들이 쉽게 이해하고 작성하기 편하게 만들어진 언어

JAVA란?

프로그래밍 언어

프로그램 실행 과정



C 언어
JAVA
Python

2진 코드

JAVA란?

객체지향언어인 JAVA

- 1991년 — Sun Microsystem사의 개발 James Gosling Patrick Naughton, Chris Warth, Ed Frank Mike Sheridna에 의해서 고안
- 1995년 — 초기에는 “Oak”로 부르던 명칭을 “JAVA”로 변경
- 1996년 — JAVA 개발 환경인 JDK1.0을 발표

초기의 JAVA는 가전제품과 같은 전자기기를 제어하기 위해 개발되었으나 World Wide Web의 발전과 함께 JAVA도 발전

- 인터넷 및 분산처리 시스템에서 효과적으로 응용프로그램을 작성
- 최근에는 프레임워크 기반의 웹 애플리케이션과 Android 운영체제의 스마트 디바이스 환경의 애플리케이션을 개발하는 데 사용

우리나라 공공 부문의 정보화 사업에 사용되는 표준화된 프레임워크가 JAVA 기반의 프레임워크

JAVA란?

객체지향 프로그래밍

명령어들의 순서에 따라 실행하는 관점이 아닌 객체의 개념으로 프로그래밍을 하는 것을 의미

각 객체들은 데이터를 저장하는 속성과 기능을 가지고 객체 서로 간에 메시지를
주고받고 처리하는 것을 의미

프로그램을 유연하고 변경이 쉽게 만들 수 있어 대규모 소프트웨어 개발과 간편한 유지보수 등의 장점

➔ 최근 웹 기반의 애플리케이션 개발에 많이 사용

JAVA란?

객체지향 프로그래밍의 특징

1 캡슐화(Encapsulation)

- 객체가 가지고 있는 정보를 은닉하여 정보를 보호하는 것을 의미
- 외부에서 객체의 정보에 직접 접근하는 것을 막고 특정 메소드를 통해 접근하도록 정의

2 정보은닉(Information Hiding)

- 외부 다른 클래스 또는 객체로부터 객체 자신의 정보를 숨기고 객체 내부에서만 접근을 허용하도록 하는 것을 의미

JAVA란?

객체지향 프로그래밍의 특징

3 추상화(Abstraction)

- 여러 객체에서 공통부분을 추출해서 정의하는 것을 의미
- 클래스의 멤버 중 공통적이고 대표성을 가지는 요소들을 뽑아 정의하는 것

4 상속성(Inheritance)

- 이미 정의된 상위 클래스(부모 클래스)의 모든 멤버들(변수와 메소드)을 하위 클래스가 물려받는 것을 의미
- 상속을 하면 상속을 받은 새로운 클래스는 변수나 메소드를 따로 정의하지 않고 상속받은 멤버들을 사용

JAVA란?

객체지향 프로그래밍의 특징

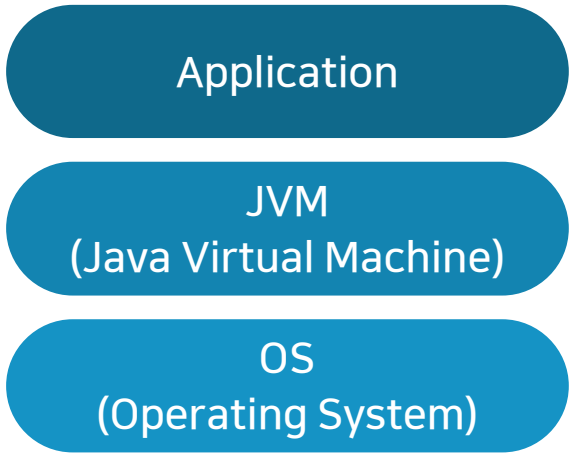
5 다형성(Polymorphism)

- 상속관계에 있는 클래스들 간에 상위 클래스 타입으로 다양한 하위 클래스 타입의 객체를 처리할 수 있도록 하는 것
- 각 객체들은 동일한 메소드 이름을 사용하지만 하위 클래스 타입 객체에 따라 각각 다른 처리

JAVA 특징

1 JAVA는 고급언어이며 컴파일언어

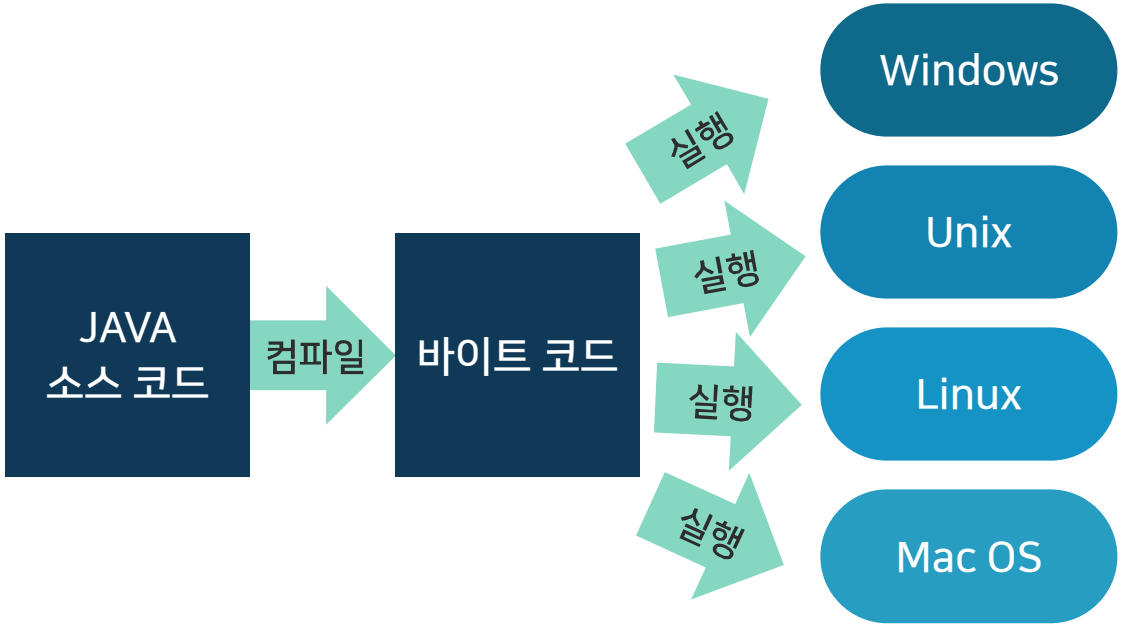
- JAVA는 고급언어이기 때문에 소스 코드 작성 후 프로그램을 실행시키기 위해서는 컴파일을 반드시 해주어야 함
  - ➔ JAVA소스를 컴파일 하면 바이트 코드로 변환
  - ➔ 컴파일 된 바이트 코드가 가상머신(JVM)에서 실행



JAVA 특징

2 JAVA는 플랫폼에 독립적인 언어

- C언어 또는 C++ 등의 언어는 운영체제 별로 코드를 작성해야 하고 각 운영체제에 맞게 컴파일 해야만 실행이 가능
- JAVA는 하나의 소스 코드로 다양한 플랫폼 또는 운영체제에 상관없이 프로그램의 실행이 가능



JAVA 특징

3 JAVA는 객체지향 프로그램 언어

- 객체의 개념으로 클래스를 사용
- 추상화, 캡슐화, 상속, 다형성 등을 이용해서 프로그래밍

4 JAVA는 멀티스레드 언어

- 하나의 프로세스에서 여러 개의 스레드를 동시에 실행할 수 있도록 프로그래밍

JAVA 특징

5 메모리관리를 자동으로 해줌

C언어 같은 고급언어

메모리의 할당과 관리를 프로그래머가
직접 해야 함

JAVA

가비지컬렉션에 의해
객체 생성 시 메모리의 생성과 소멸의
생명주기를 가상머신(JVM)이 해줌

→ 프로그래머는 핵심 로직 구현에만
집중할 수 있음

JAVA
개발환경 구축

COMPILER
DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

개발환경 구축과 JDK

개발환경 구축

JAVA언어를 기반으로 프로그래밍을 하기 위해서는 JAVA 개발환경을 제공하는 개발도구인 JDK를 이용하여 개발해야 함

개발환경의 구축과정

JDK 설치 → 환경 변수 설정 → Eclipse 설치

개발환경 구축과 JDK

JDK의 구성

JDK는 JAVA 언어 기반의 프로그래밍을 할 수 있는 개발 환경을 제공함

JAVA 프로그램을 만들기 위해서는 JDK 설치 및 프로그래밍을 해야 함

JDK = JAVA 개발도구 (컴파일러, 툴) + JVM (가상머신) + JAVA API (라이브러리)

개발환경 구축과 JDK

JDK의 구성

1 JAVA API(Application Programming Interface)

- JAVA 프로그램개발에 사용되는 라이브러리들의 집합
- 이 라이브러리에는 클래스와 인터페이스 계층 구조로 구성되어 있음

2 JAVA 가상 머신(JVM: Java Virtual Machine)

- JAVA가 실행될 수 있는 플랫폼 환경
- 가상머신은 운영체제에 설치되어 있고 JAVA 소스 코드를 컴파일 한 바이트 코드를 실행시키는 역할을 함

개발환경 구축과 JDK

JDK의 종류

1 Java SE: Java Platform, Standard Edition

- 표준 JAVA 플랫폼으로 JAVA 가상 머신 규격 및 API를 포함하고 있음

2 Java EE: Java Platform, Enterprise Edition

- Java SE에 웹 어플리케이션 서버(Web Application Server)에서 동작하는 기능을 추가한 플랫폼

3 Java ME: Java Platform, Micro Edition

- 휴대전화, PDA 등 작은 단말기나 작은 가전제품에 JAVA를 지원하기 위해 만든 플랫폼

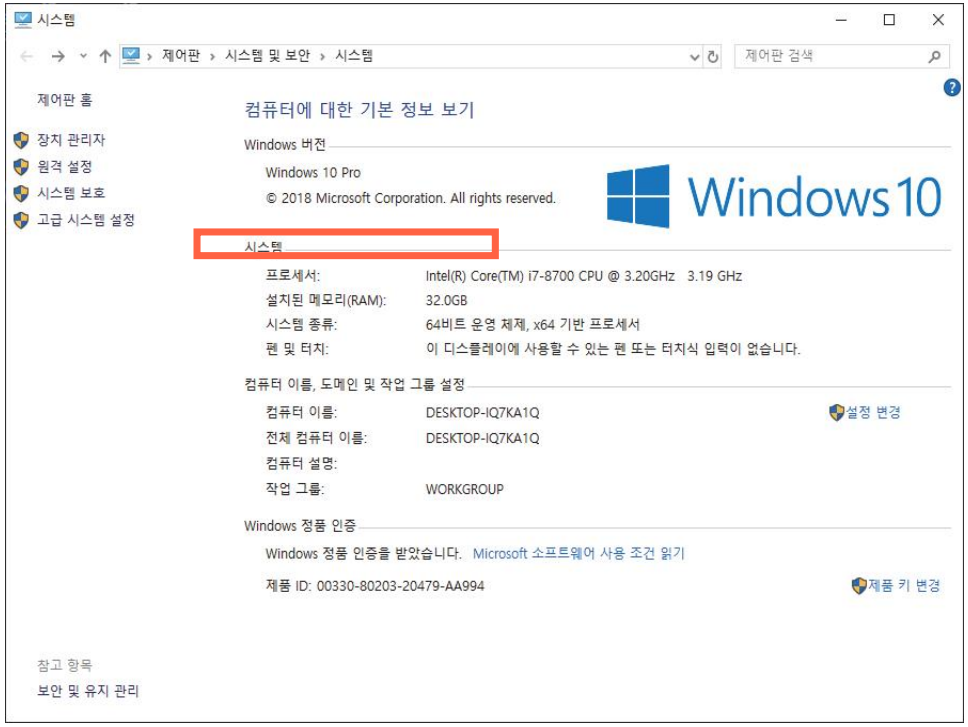
JDK 설치

운영체제 확인

JDK 다운로드에 앞서 설치할 컴퓨터의 운영체제를 먼저 확인한 후, 운영체제에 맞는 JDK를 다운받아 설치

운영체제 확인 방법

- 제어판 → 시스템 및 보안 → 시스템에 접속
- 파일탐색기 → 내 PC의 오른쪽 버튼을 누르고 속성을 선택

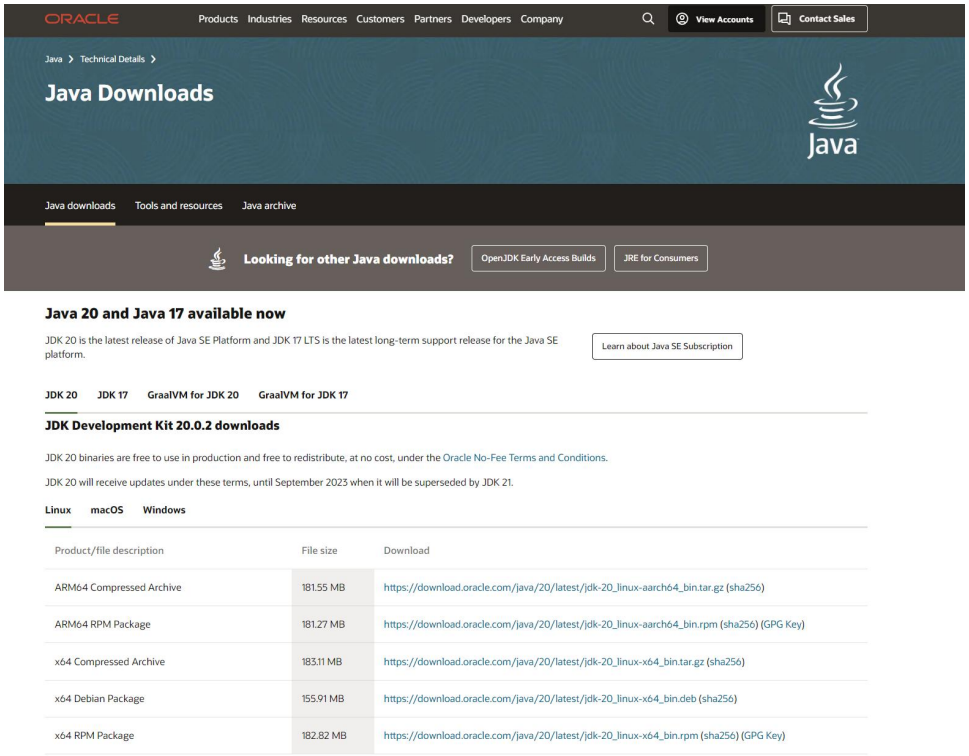


JDK 설치

JDK 다운로드

Oracle 사이트에서 접속해 JDK를 다운로드 함

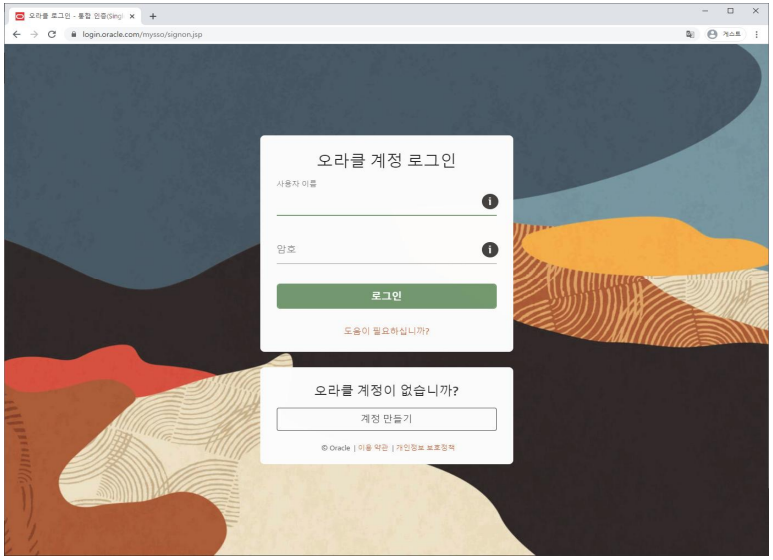
https://www.oracle.com/technetwork/java/javase/downloads/index.html



JDK 설치

JDK 다운로드

4 다운로드를 위해 Oracle 계정으로 로그인 하고 다운로드



JDK 설치

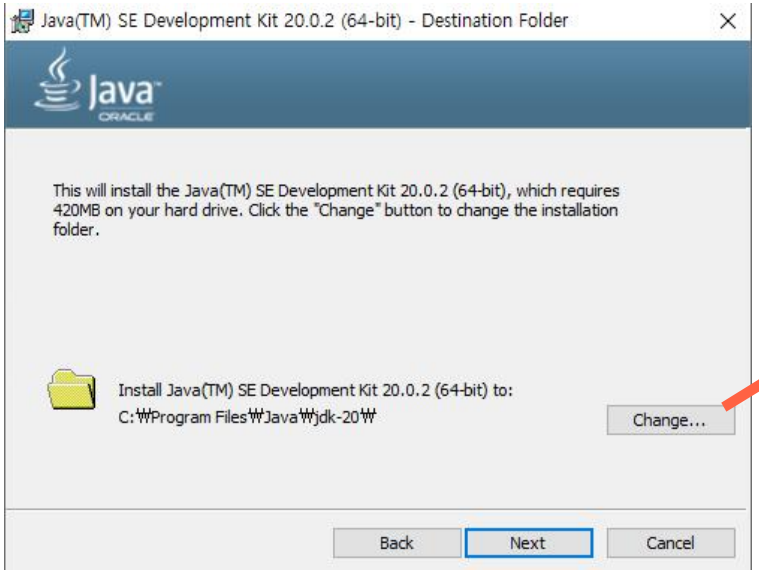
Java SE 8 JDK 설치

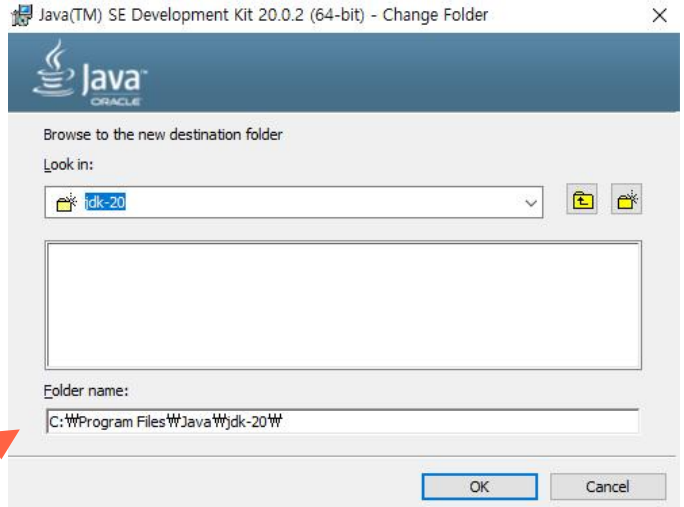
1 다운로드 한 JDK 파일을 실행

JDK 설치

Java SE 8 JDK 설치

2 설치할 폴더 설정





JDK 설치

Java SE 8 JDK 설치

3 설치 완료

JDK 설치

윈도우 환경 변수 설정

제어판 → 시스템 및 보안 → 시스템 창 열고 고급 시스템 메뉴 클릭

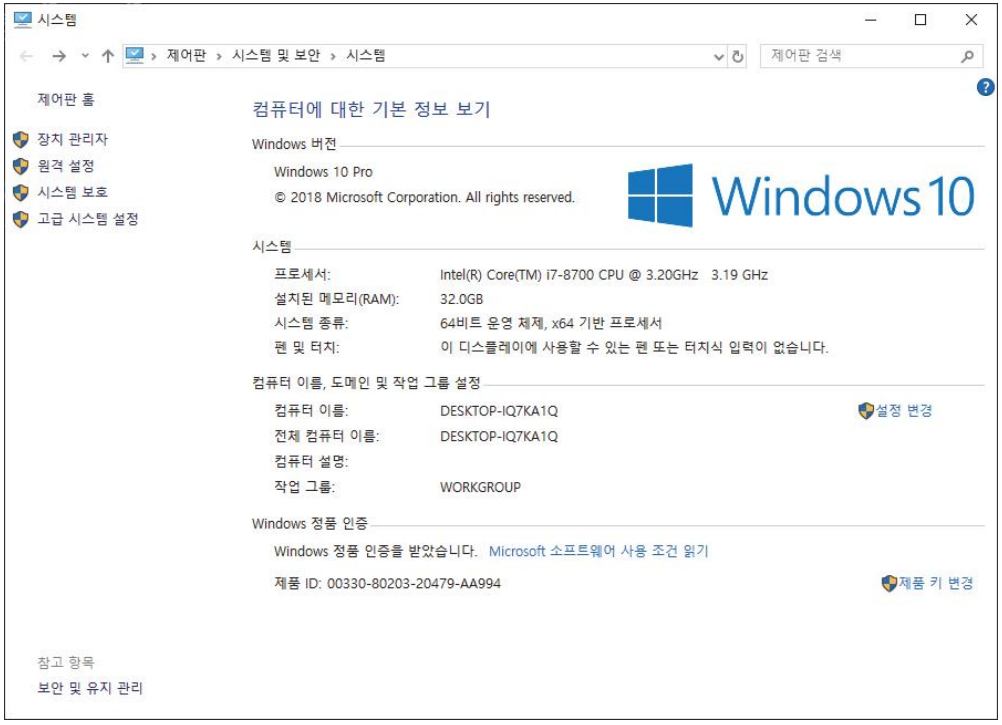
또는

시스템 환경설정 창 열기

또는

Windows키 + Pause Break 키를 눌러
시스템 속성 설정 창 열기

환경 변수	운영체제가 프로그램을 실행할 때 프로세스를 생성하면서 참조하는 변수

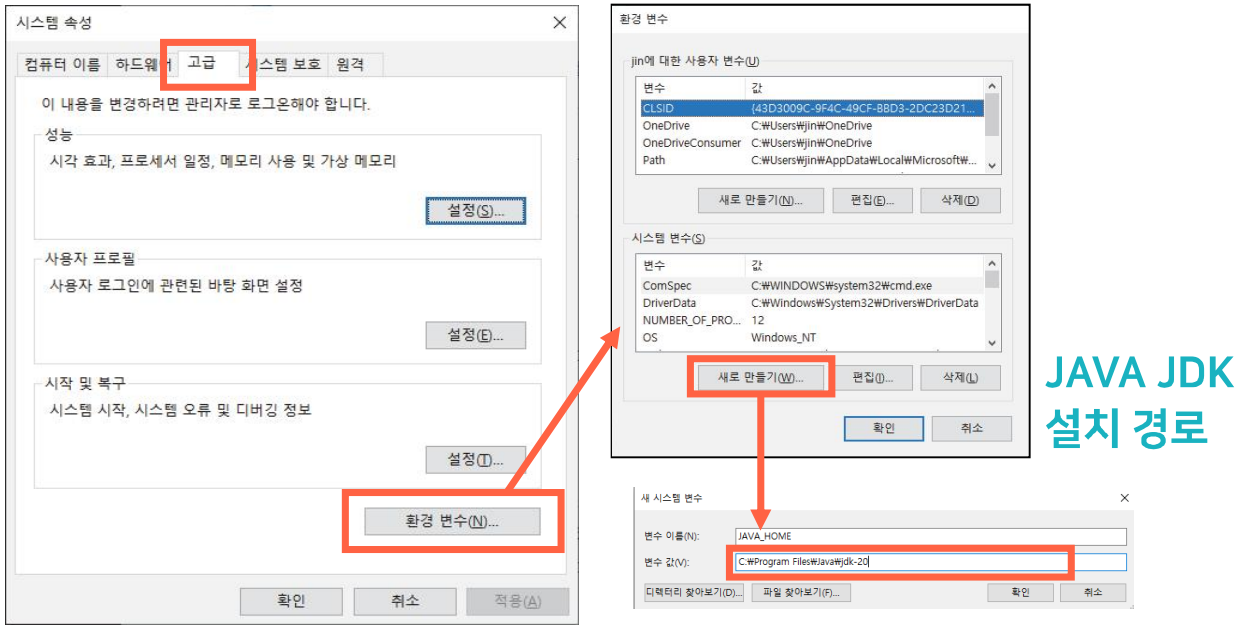


JDK 설치

윈도우 환경 변수 설정

JAVA_HOME 새로 만들기

시스템 속성 ➔고급 탭 ➔ 환경 변수 클릭 ➔환경 변수 설정 창
➔ 시스템 변수 JAVA_HOME 새로 만들기



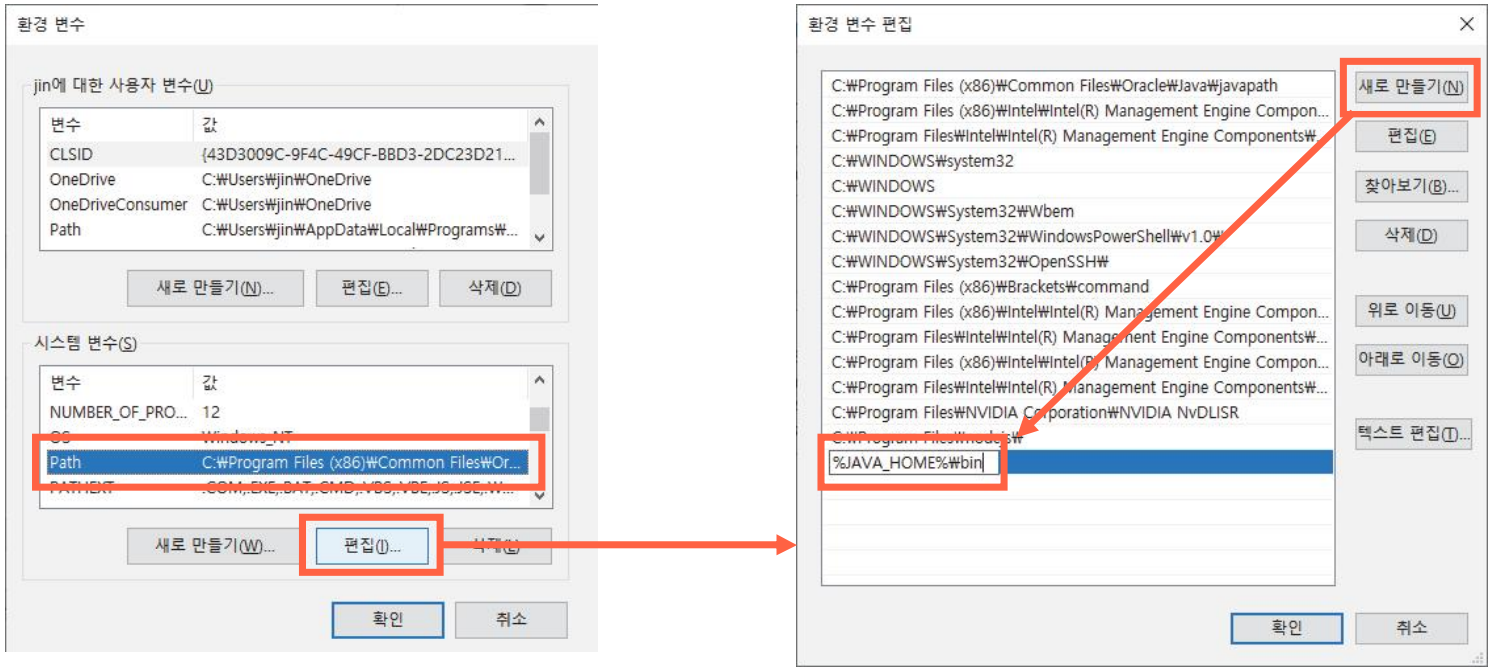
JAVA JDK
설치 경로

JDK 설치

윈도우 환경 변수 설정

JAVA_HOME 새로 만들기

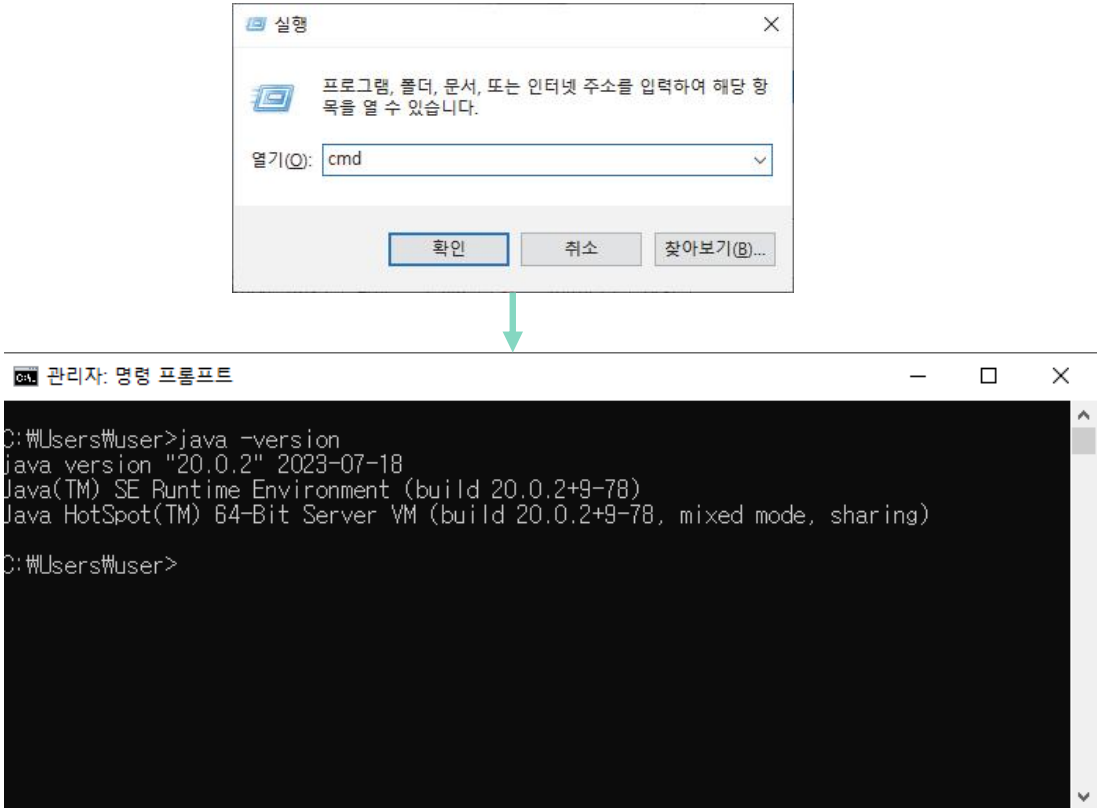
환경 변수 설정 창 → 시스템 변수 path 수정 → %JAVA_HOME%₩bin 추가



JDK 설치

Java SE 8 JDK 설치 확인

cmd 에서 Java –version 명령으로 설치되어 있는 Java 버전 확인



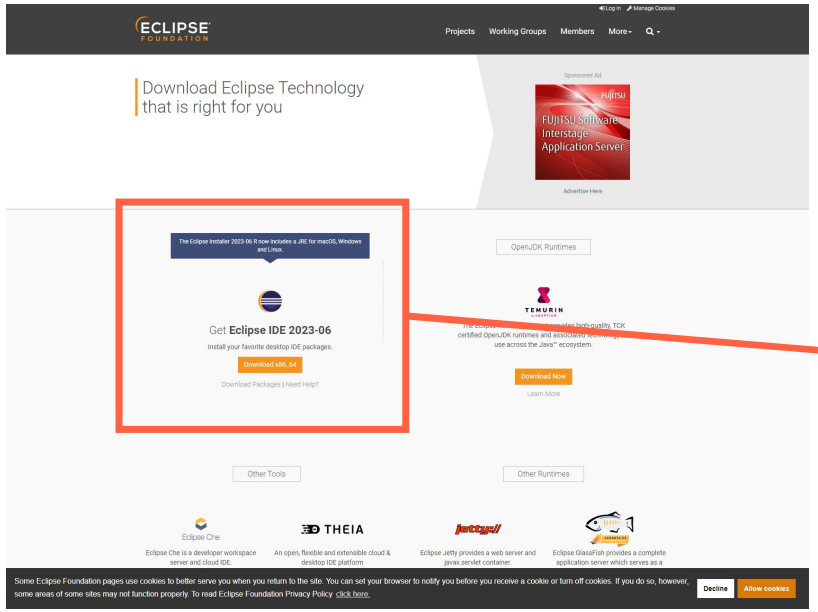
이클립스 설치와 실행

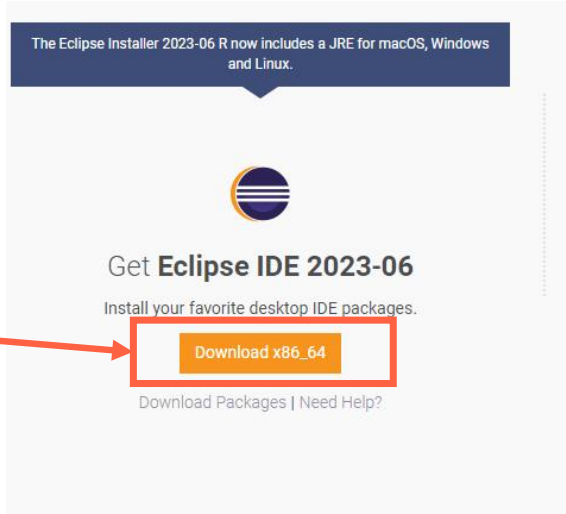
이클립스 설치

이클립스 다운로드

JAVA 개발을 위한 Java Developers 버전을 다운로드 함

https://www.eclipse.org/downloads/



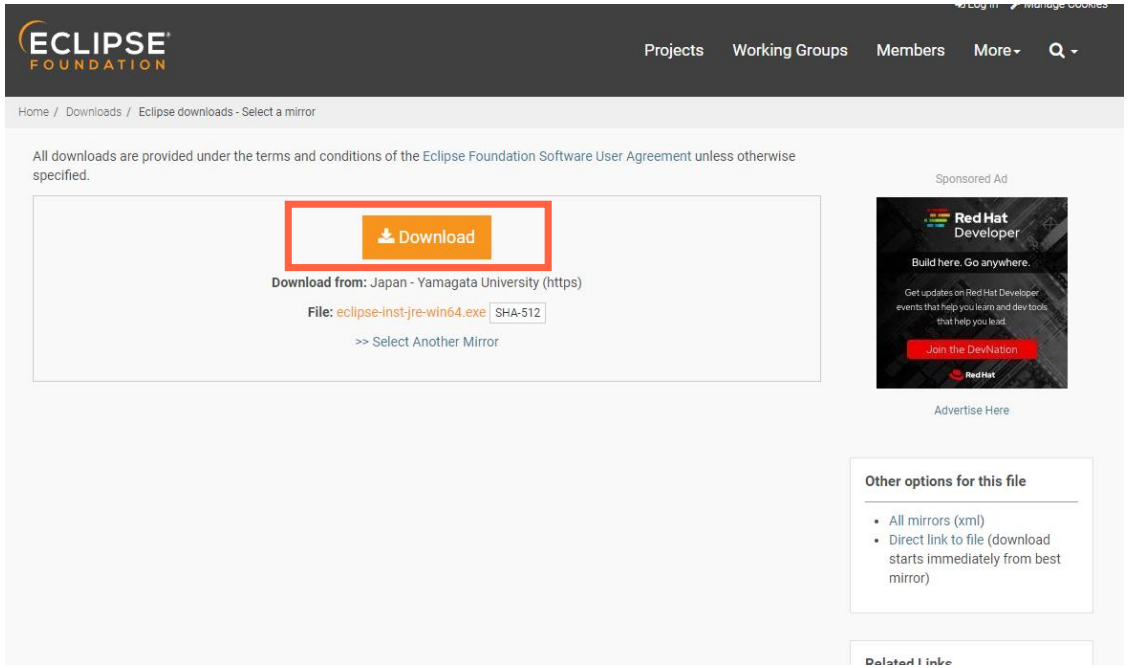


이클립스 설치와 실행

이클립스 설치

이클립스 다운로드

Eclipse 다운로드

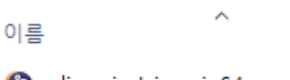


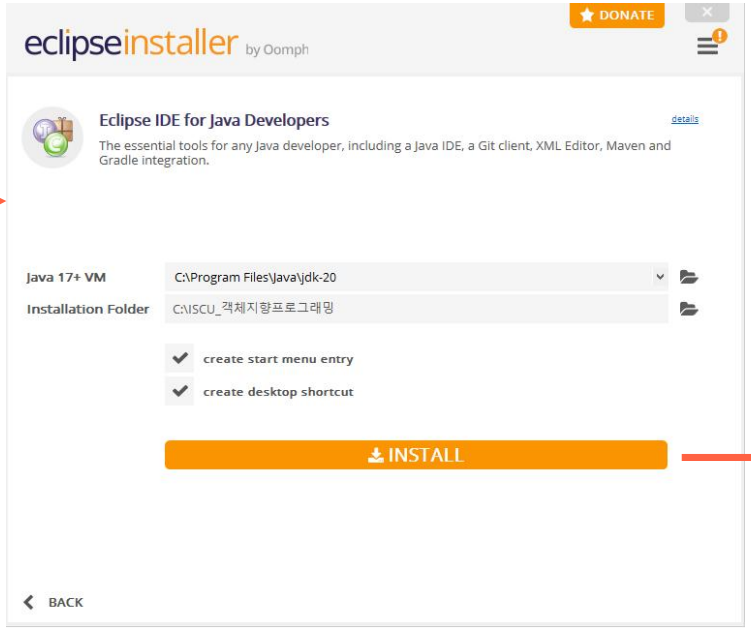
이클립스 설치와 실행

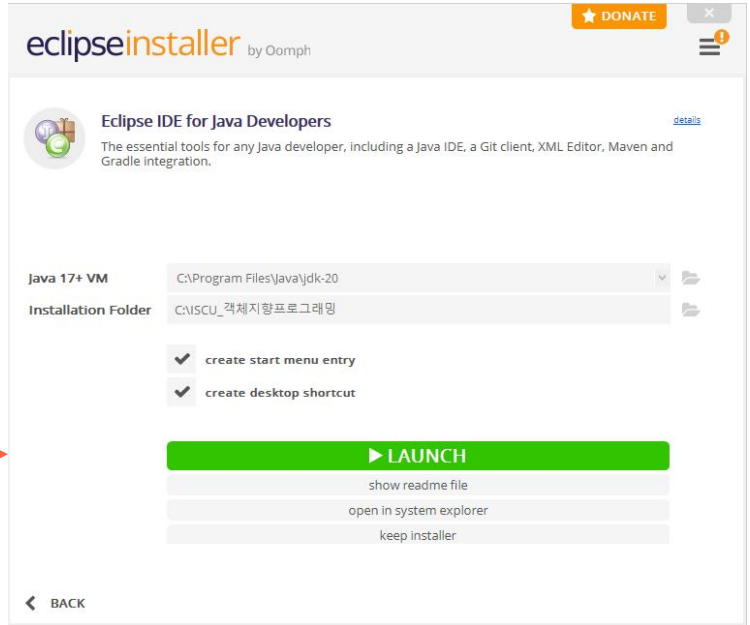
이클립스 설치

이클립스 다운로드

다운로드 한 Eclipse 설치



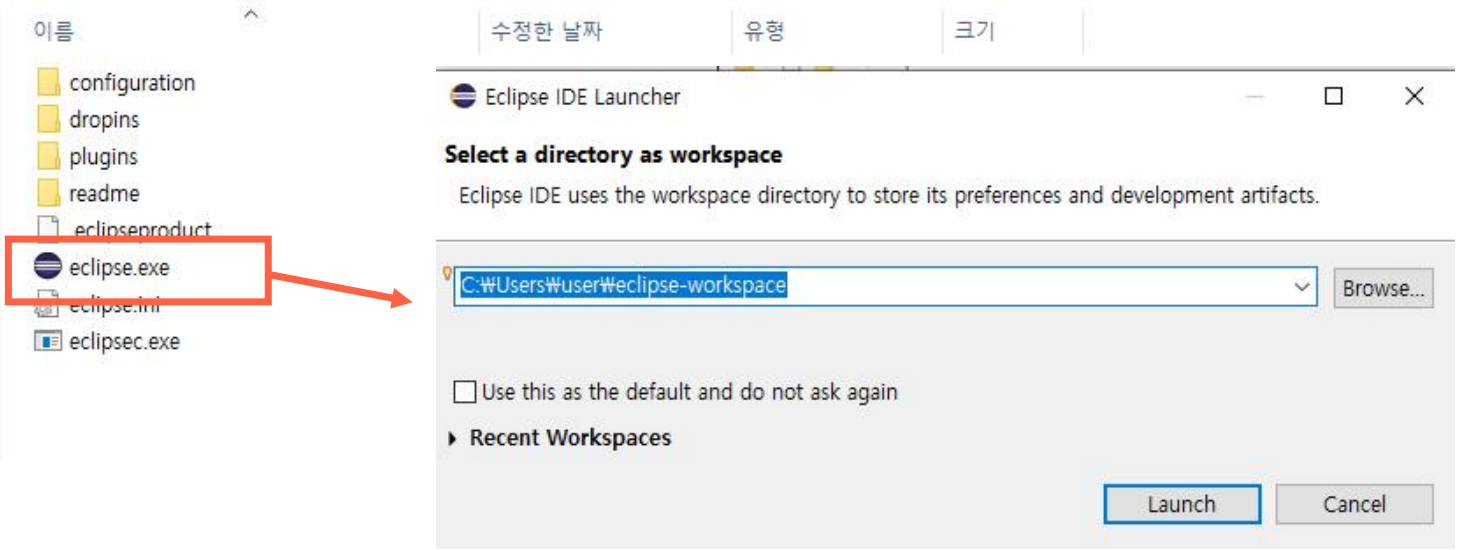




이클립스 설치와 실행

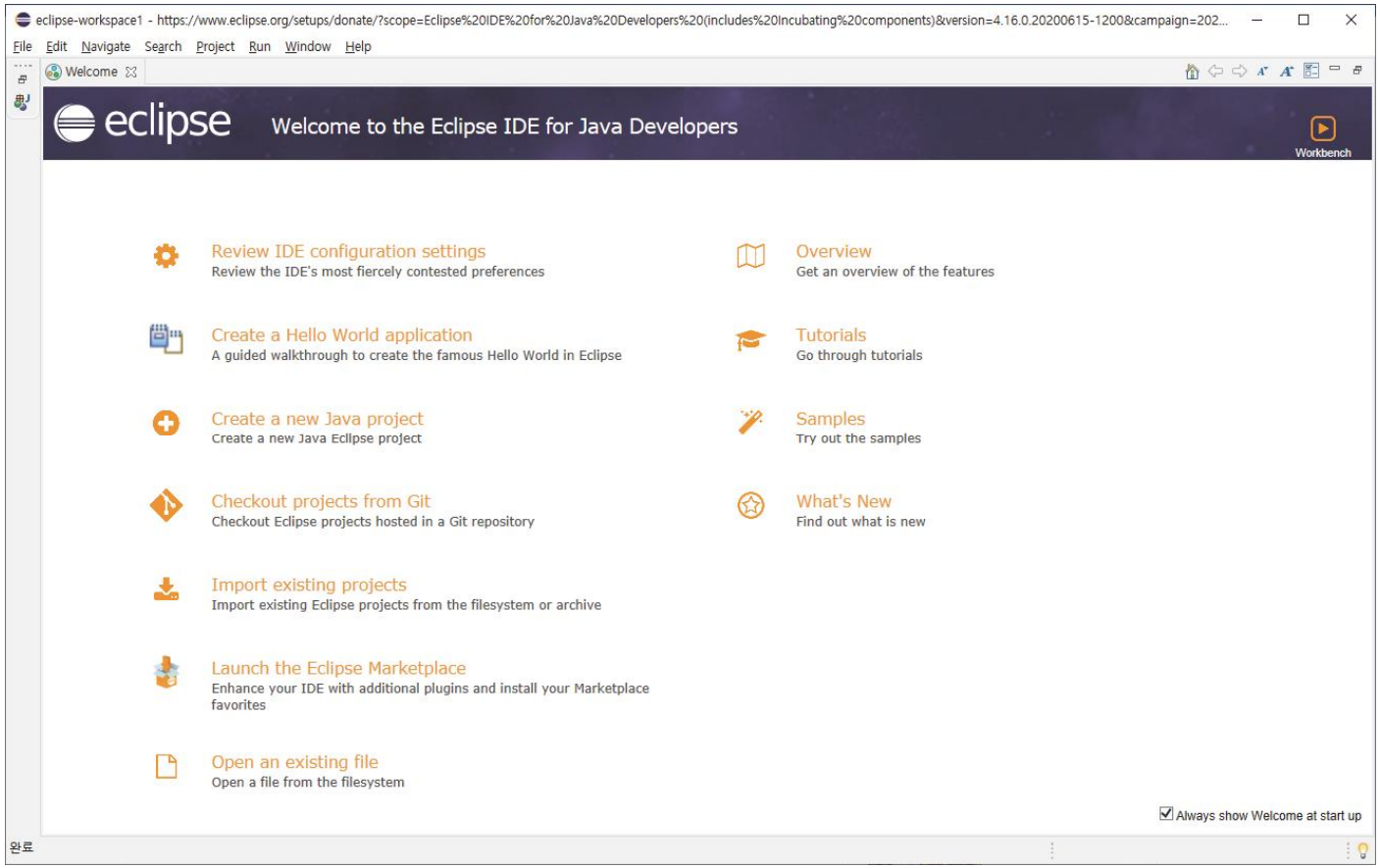
이클립스 실행

eclipse.exe 실행 → Workspace 폴더 설정 창



이클립스 설치와 실행

이클립스 실행



JAVA 프로그래밍 처리과정

일반 vs JAVA 프로그래밍 처리과정

프로그래밍 처리과정

일반적인 고급언어 기반의 프로그래밍 과정

프로그램 소스 코드 작성

↓

컴파일러

↓

기계어 2진 코드

↓

CPU 실행

일반 vs JAVA 프로그래밍 처리과정

프로그래밍 처리과정

일반적인 고급언어 기반의 프로그래밍 과정

소스 코드를 작성하고 컴파일러를 이용해서 컴파일 한 후,
컴파일 된 2진 코드를 CPU가 알고 실행함

컴파일	프로그래머가 작성한 코드를 컴퓨터의 CPU가 해석할 수 있는 2진 코드로 변경하는 것

일반 vs JAVA 프로그래밍 처리과정

JAVA 프로그래밍 처리과정

다른 언어와 다르게 컴파일이 되면 바로 기계어로 변환되는 것이 아니라 바이트 코드로 컴파일해서 JVM(가상머신)에서 실행함

JAVA 프로그램 소스 코드 작성
First.java

→

컴파일러
Javac.exe

↓

바이트 코드
First.class

←

JVM에서 실행
JAVA.exe

JAVA프로그램 구조 살펴보기

JAVA프로그램 구조

클래스 구조

클래스

클래스 이름

메소드 이름

메소드

```
public class FirstJava {
    public static void Main(Sting[] args) {

        System. out. PrintIn("Hello~ JAVA~!!");

    }
}
```

- 데이터
- 키워드
- 상수
- 기호
- 식별자
- 주석

JAVA프로그램 구조

클래스 구조

1 데이터

- 숫자(정수, 실수), 문자, 문자열, 논리값 등

2 키워드

- JAVA 프로그램 내부에서 이미 약속된 단어로 이루어진 명령어

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

JAVA프로그램 구조

클래스 구조

3 JAVA 내부에서 정의되어 있는 상수

- true, false, null

4 JAVA에서 사용하는 기호

- 대괄호, 중괄호, 소괄호, 마침표, 쉼표, 세미콜론 등

JAVA프로그램 구조

클래스 구조

5 식별자

- JAVA 프로그램 내부에서 식별하기 위한 이름
- 클래스의 이름, 메소드의 이름, 변수의 이름이 식별자에 해당함

식별자 작성 방법	→ 식별자를 작성할 때 키워드를 사용할 수 없음 → 식별자는 대 · 소문자를 철저하게 구분함 → 클래스나 메소드가 어떤 작업을 수행하는지 의미를 부여하는 이름으로 작성함

JAVA프로그램 구조

클래스 구조

6 주석

JAVA 소스 코드에서 프로그램에 수행되는 코드가 아닌 코드 내부에 프로그래머가 남기는 메모와 같음

주석 작성 방법

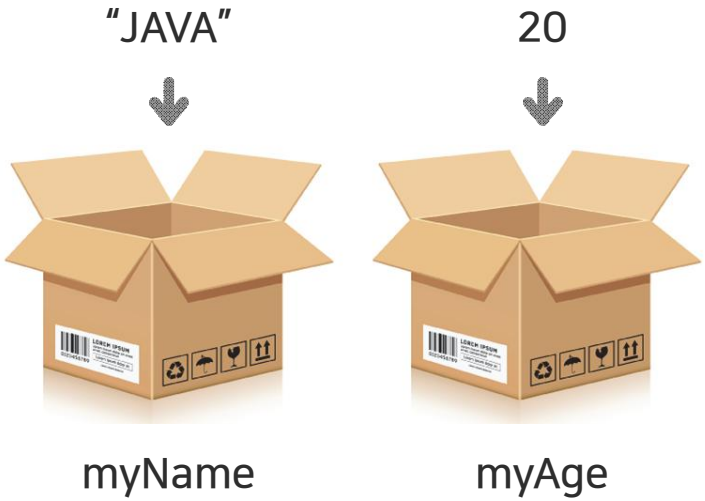
```
//  한의 행을 주석 처리할 때는 행 맨 앞에 슬래시 두 개를 기술해 주면 됩니다.
/*
   여러 행의 주석은 앞에 /* 로 시작하고 마지막에 */ 로 마감해 주면 됩니다.
*/
/*
   사용자 이름과 사용자 나이를 저장하는 변수를 선언
*/
String name = "KING"; // 사용자 이름
int age; // 사용자 나이
```

JAVA프로그램 구조

변수

하나의 데이터를 저장하고, 변수에 접근할 수 있도록 주소의 기능을 가짐

변수 선언	변수를 사용할 때, 어떤 데이터를 저장할 것인지 결정해서 변수를 정의하는 것



변수 선언 방법

데이터 자료형 변수이름;

↓

int age;

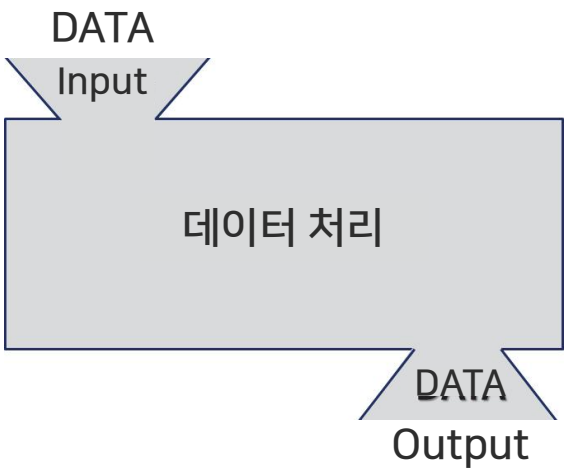
↓

정수 데이터를
저장하는 변수 age

JAVA프로그램 구조

메소드

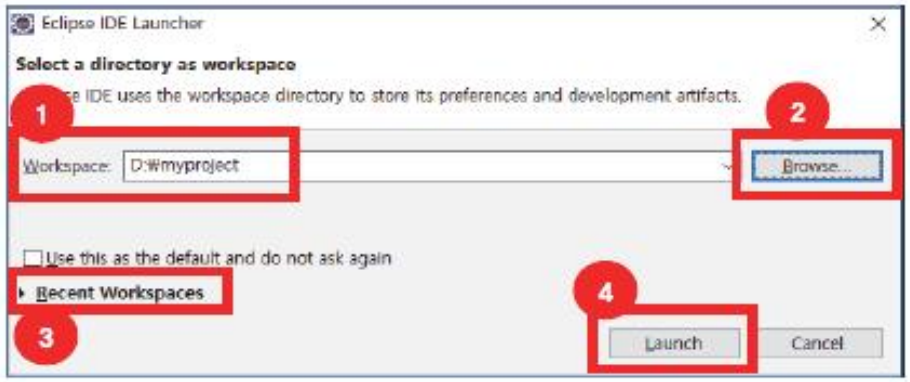
함수	특정 목적의 기능을 수행하는 코드들의 집합
메소드	객체에 정의된 함수 → JAVA에서는 함수가 객체 내부에서만 존재하기 때문



이클립스로 JAVA 프로그램 작성하기

workspace 설정

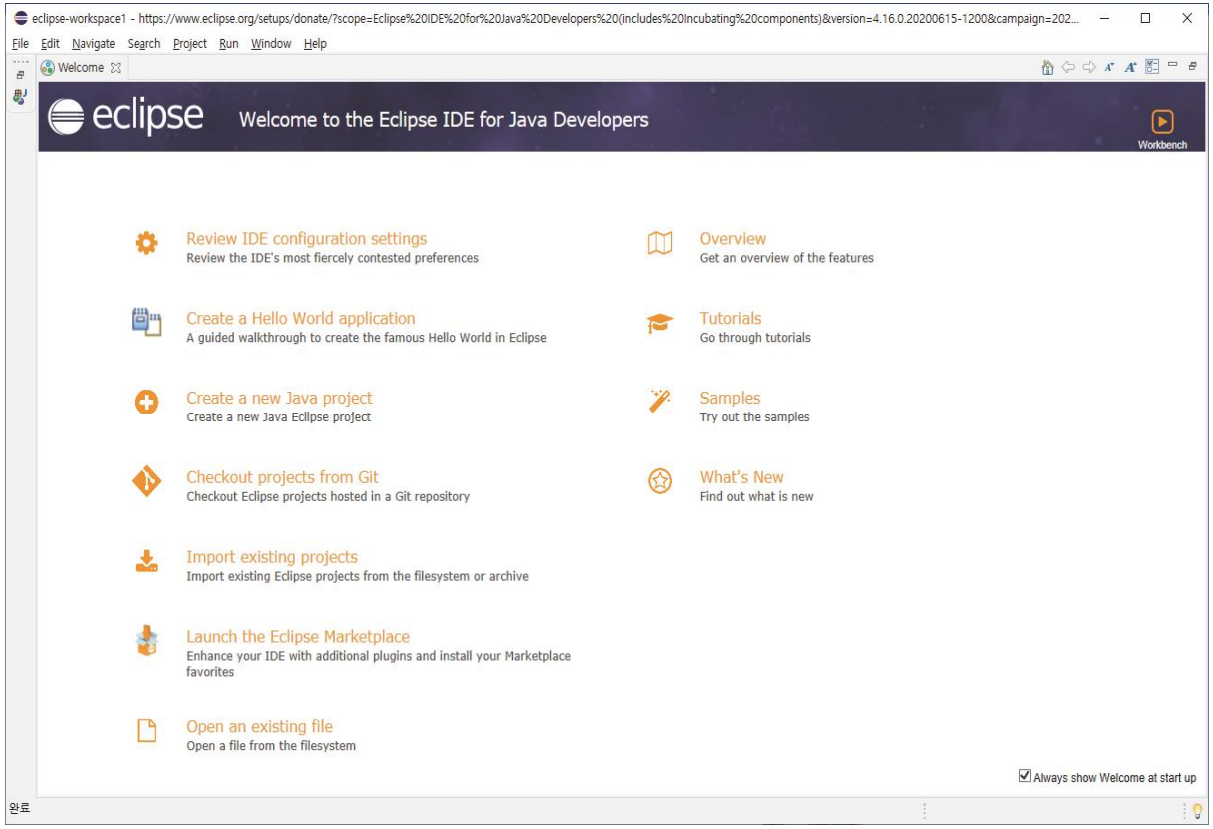
이클립스를 실행하면 먼저 workspace 설정하는 창이 뜸



1. workspace 경로
2. 새로운 workspace 경로를 찾아 설정
3. 최근 workspace 경로를 나타냄
4. 이클립스 프로그램을 실행

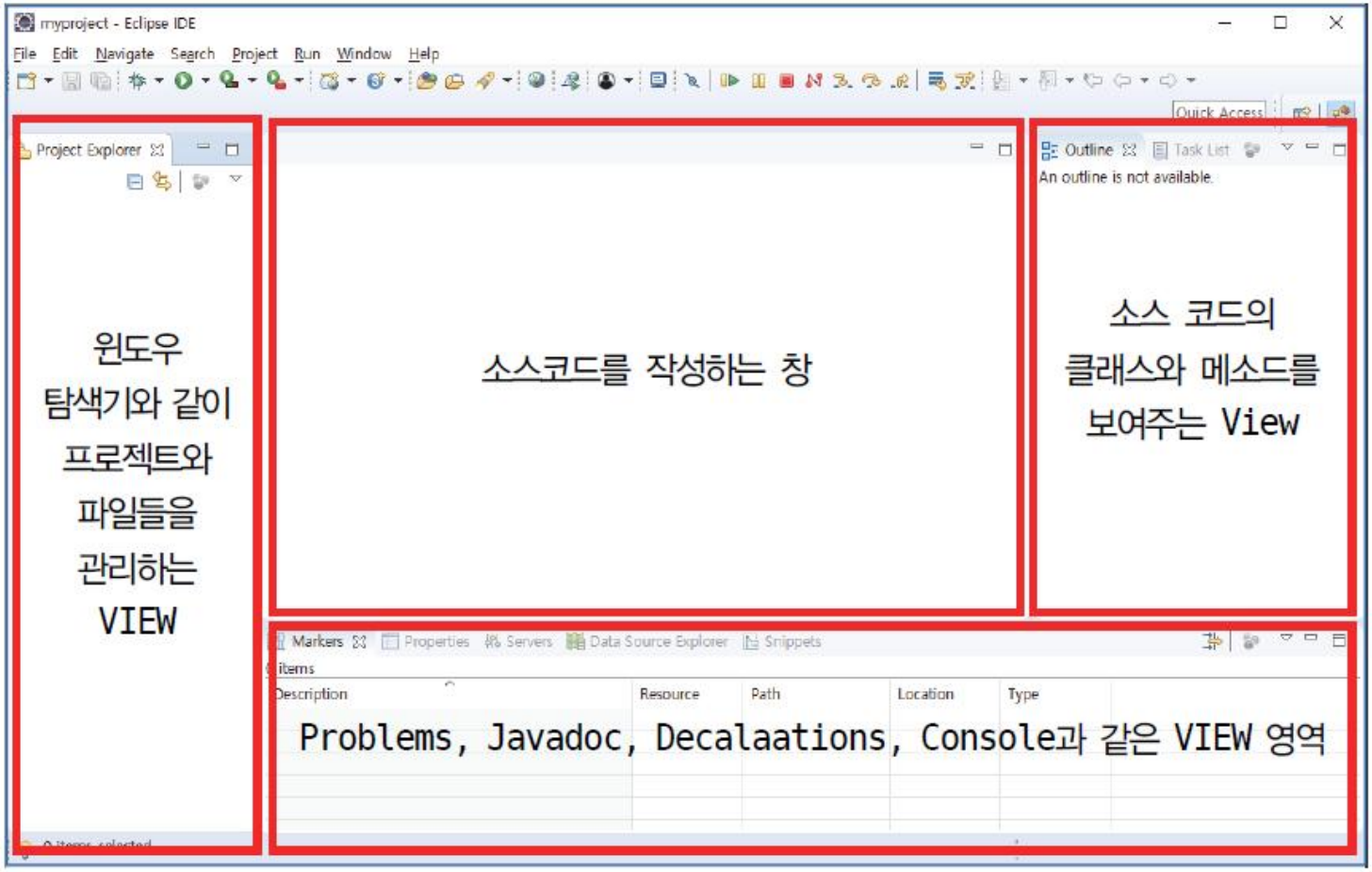
이클립스로 JAVA 프로그램 작성하기

이클립스 실행 화면



이클립스로 JAVA 프로그램 작성하기

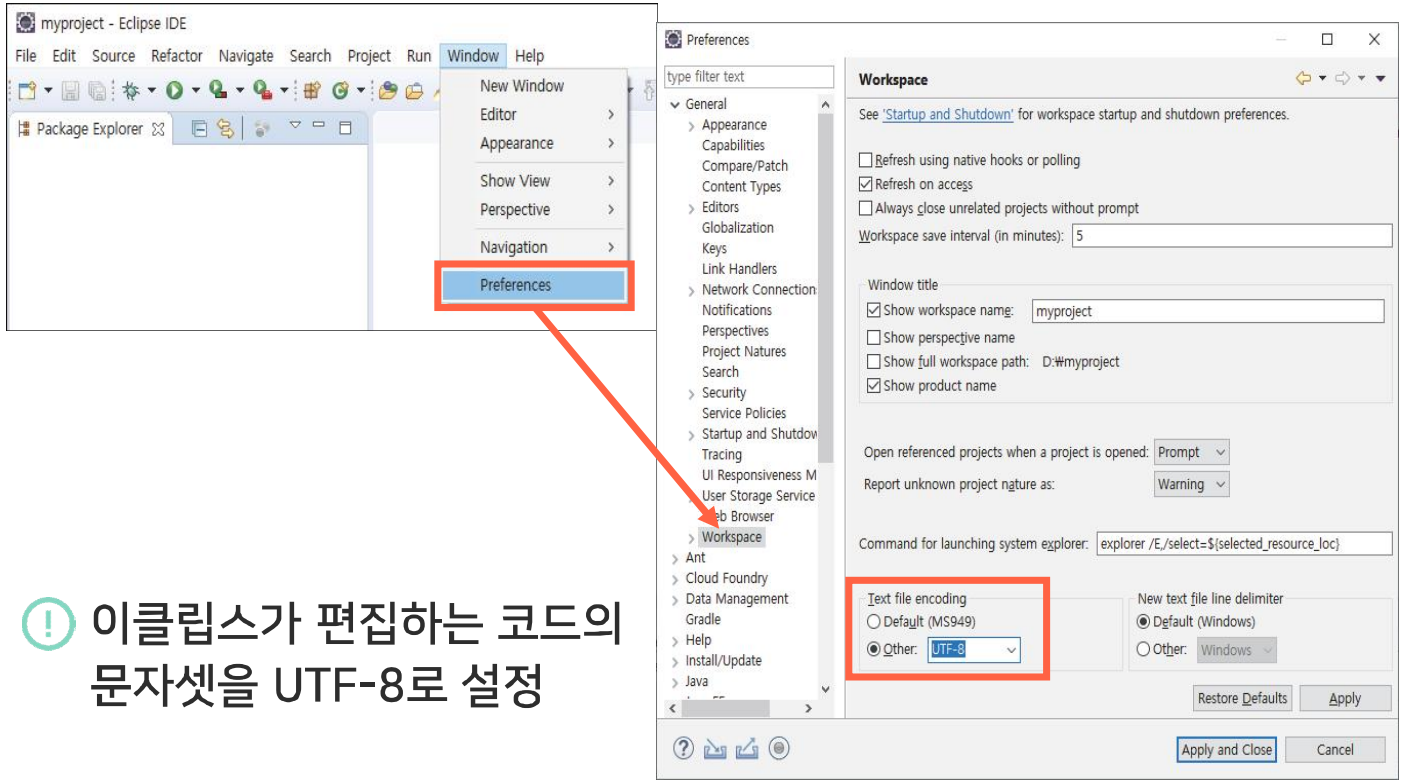
이클립스 영역별 화면 구분



이클립스로 JAVA 프로그램 작성하기

이클립스 환경설정

Windows → Preferences 메뉴 선택

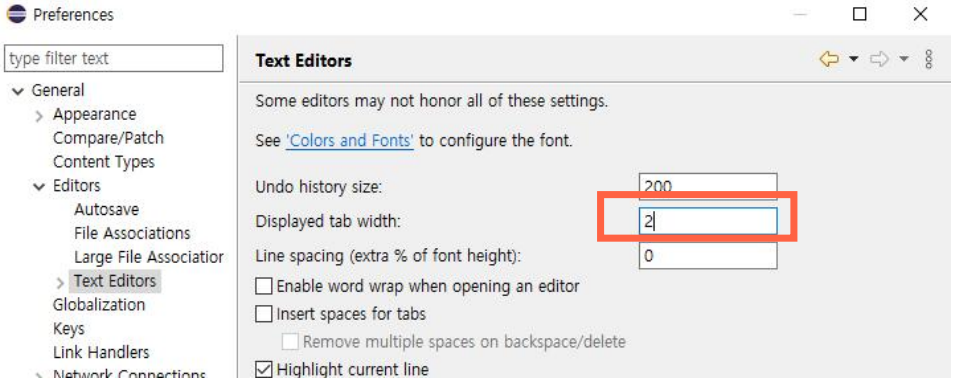


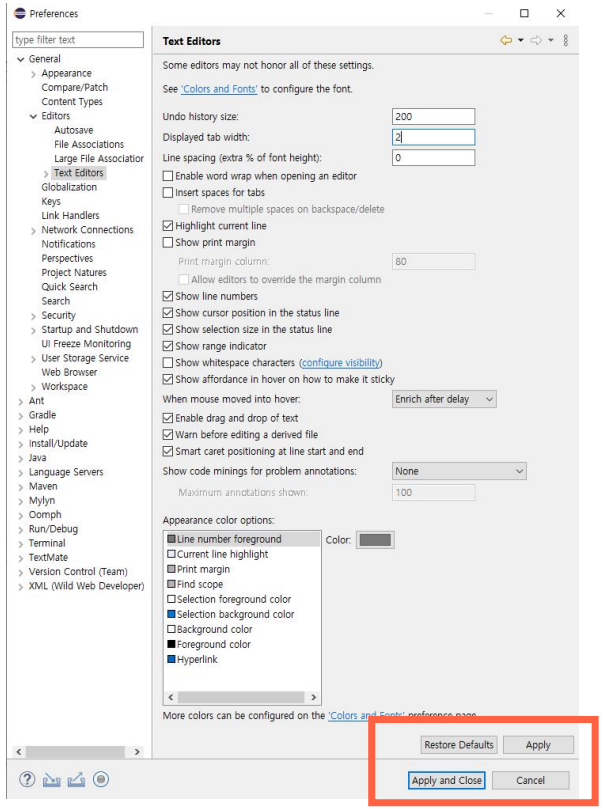
이클립스가 편집하는 코드의 문자셋을 UTF-8로 설정

이클립스로 JAVA 프로그램 작성하기

코드 들여쓰기 설정

Windows → Preferences → General → Editor → Text Editor 선택

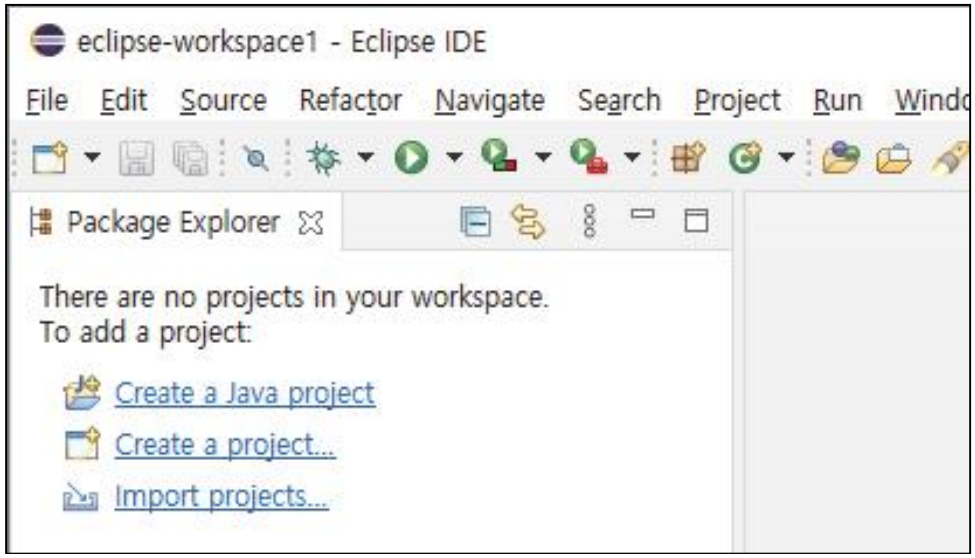




이클립스로 JAVA 프로그램 작성하기

프로젝트의 생성

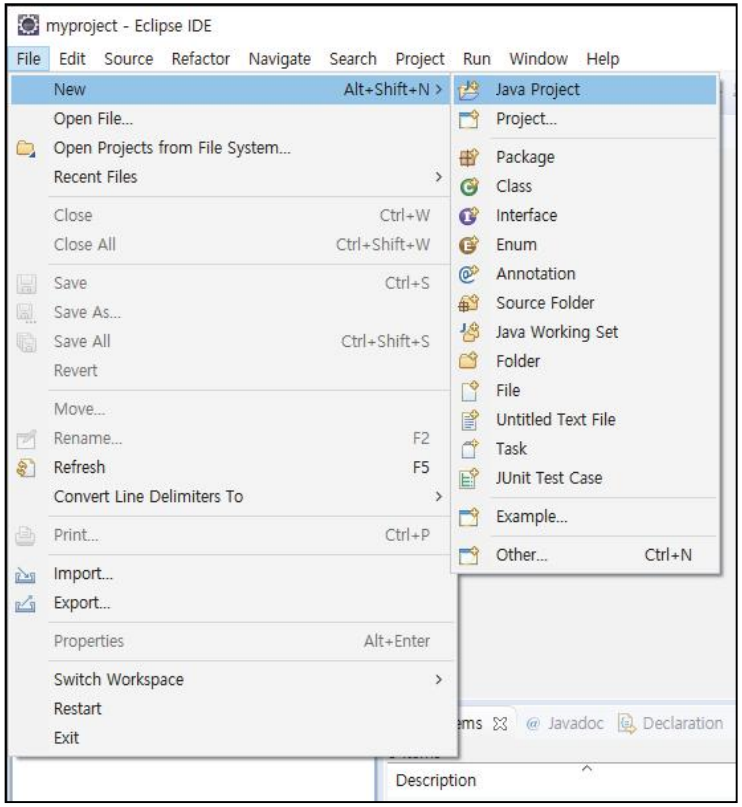
방법1 첫 화면의 Create a java Project를 클릭해서 JAVA 프로젝트 생성



이클립스로 JAVA 프로그램 작성하기

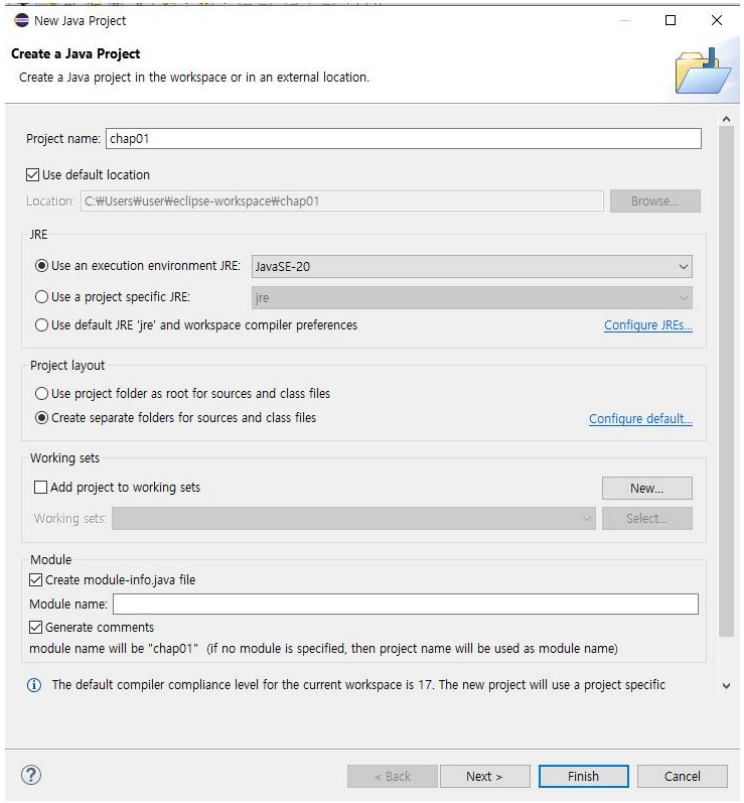
프로젝트의 생성

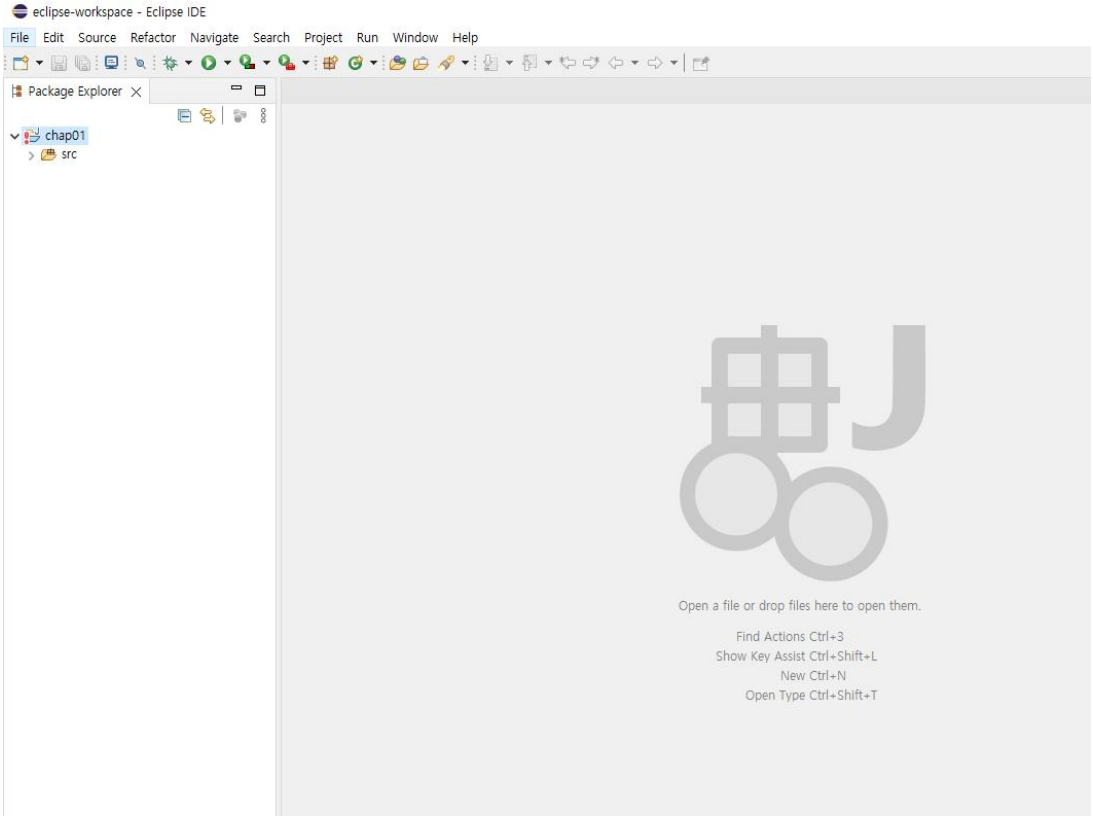
방법2 File→New→Java Project 메뉴 선택



이클립스로 JAVA 프로그램 작성하기

프로젝트 생성과 설정

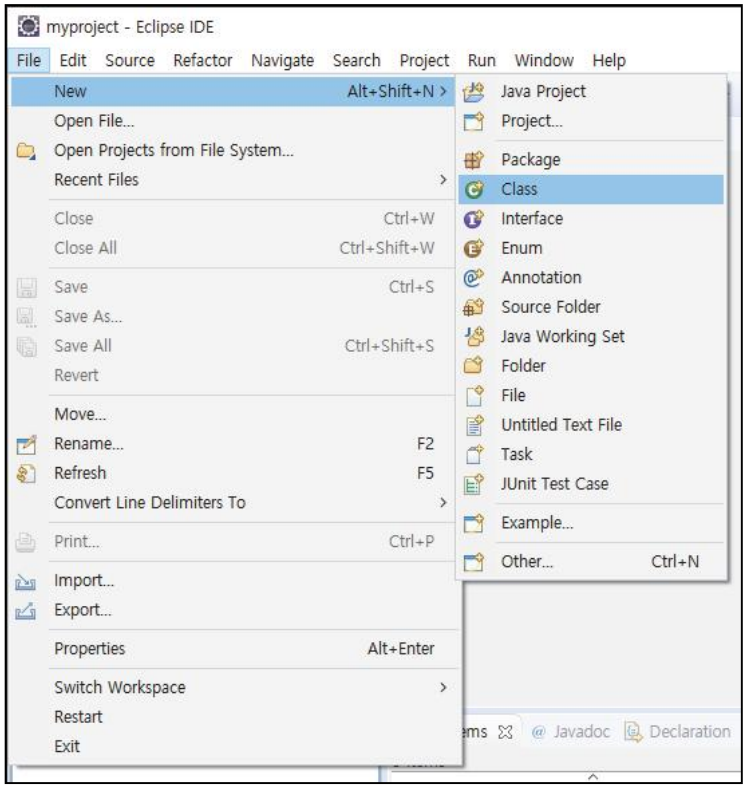




이클립스로 JAVA 프로그램 작성하기

클래스 파일 생성

File → New → Class 메뉴 클릭

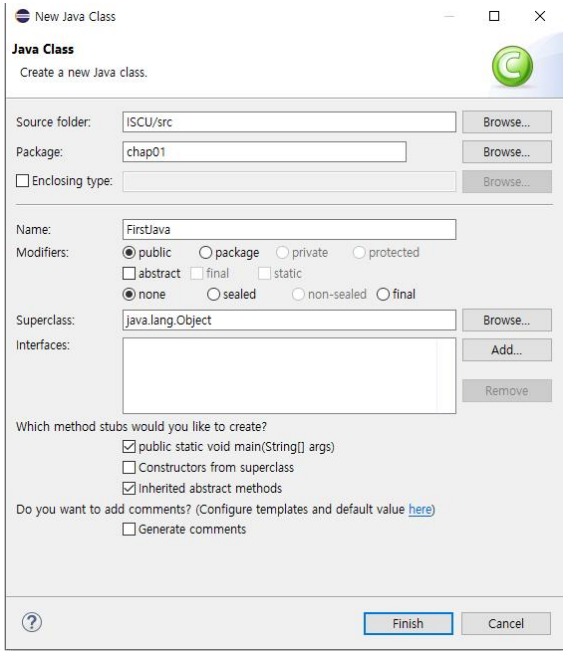


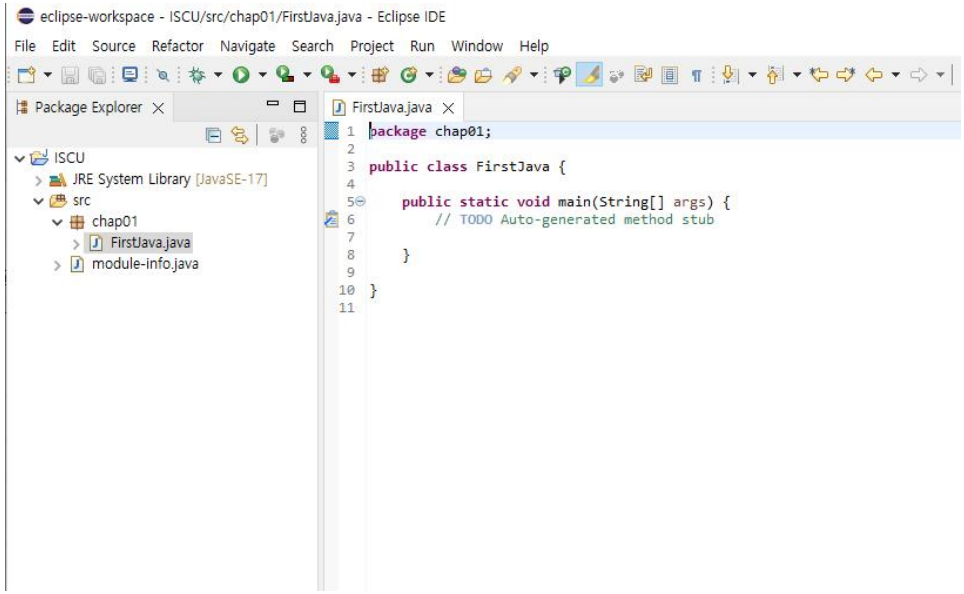
이클립스로 JAVA 프로그램 작성하기

클래스 파일 생성 설정

Package 설정과 Name 설정

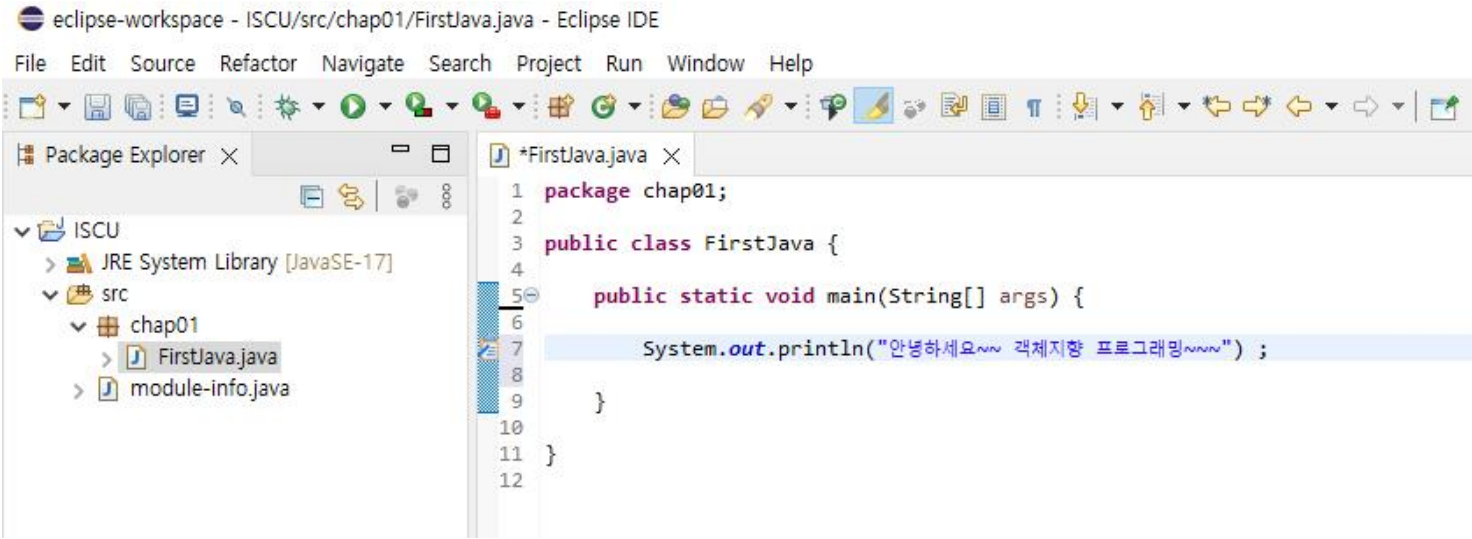
그 아래 public static void main(String[] args) 체크





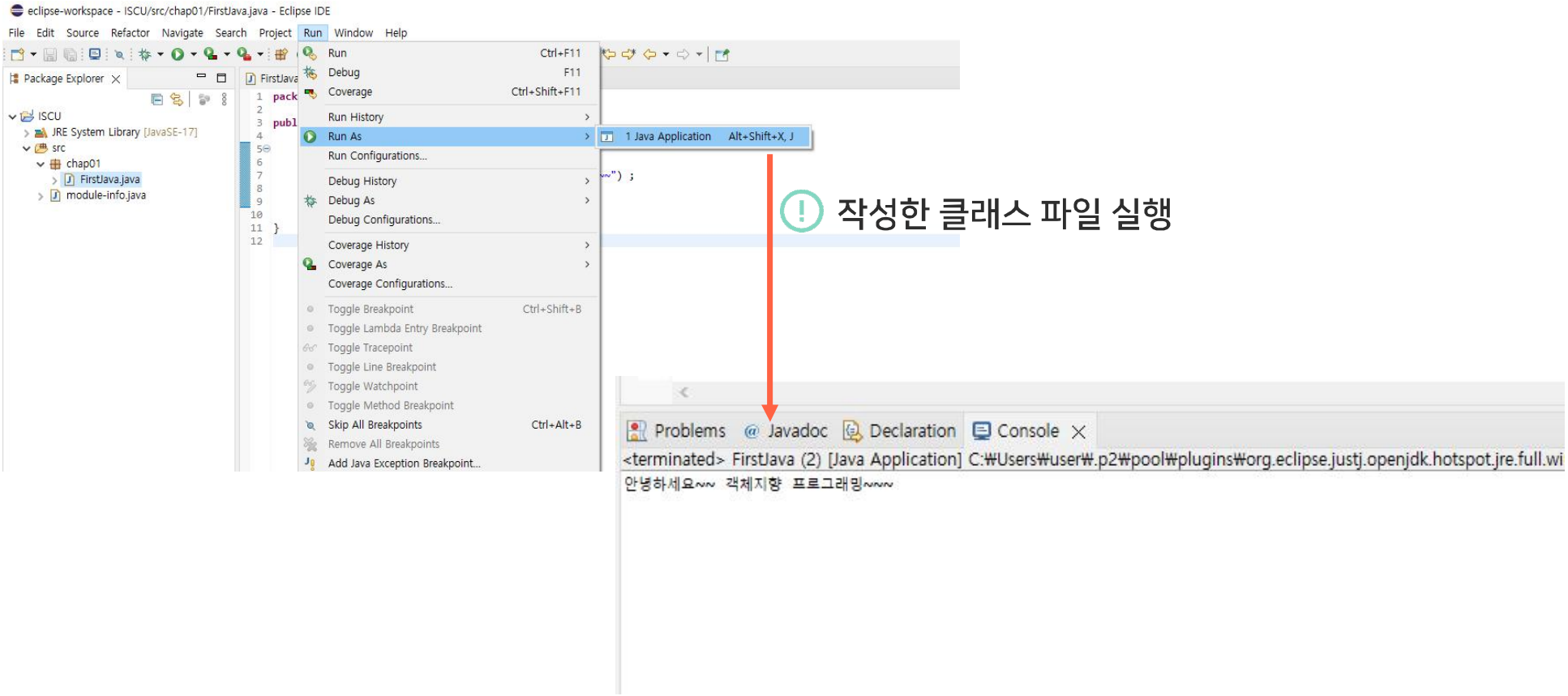
이클립스로 JAVA 프로그램 작성하기

코드 작성 및 편집



이클립스로 JAVA 프로그램 작성하기

java 파일 실행과 결과 확인



END

메소드

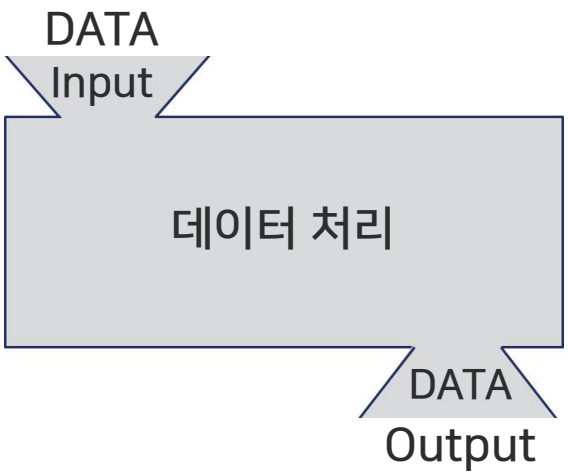
COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

메소드 정의와 예제

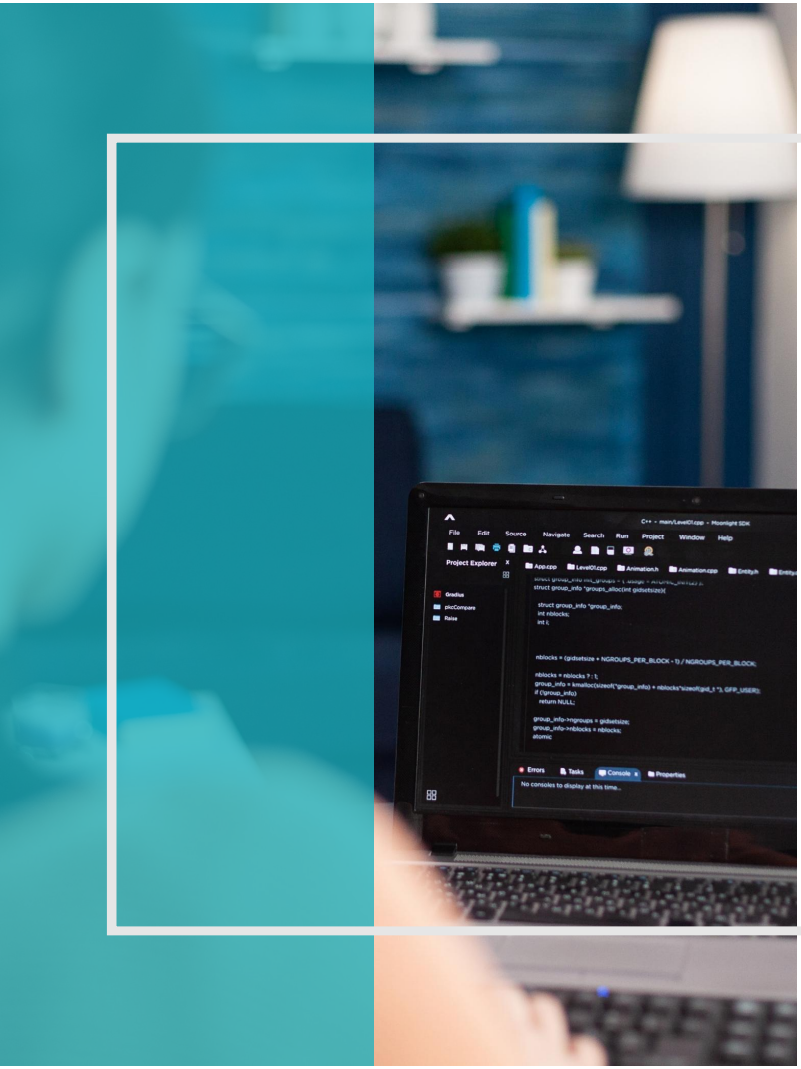
메소드의 의미

필요한 데이터를 받아 처리한 후, 결과를 반환함

데이터를 받아 처리할 수도 있지만, 데이터를 받지 않고 처리할 수도 있고, 데이터 처리 결과를 반환하는 경우와 반환하지 않는 경우도 있음



메소드 정의와 예제



메소드 정의 방법

① 반환 데이터 타입
② 메소드 이름(③데이터 타입 변수 명, 데이터 타입 변수 명, ...)

```
{
    /* ④ 처리할 문장들*/
}
```

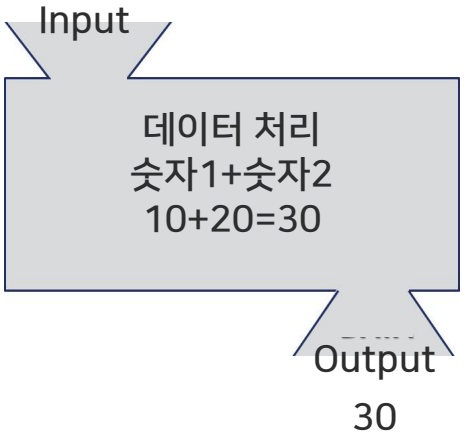
메소드 정의와 예제

메소드 예제1

```
// 선언 부
// 반환 데이터 타입은 long
// 메소드 이름은 add
// 매개변수 long n1, long n2로 정의

long add(long n1, long n2) {
    long result = n1 + n2; // 매개변수의 덧셈연산
    return result; // return 키워드는 값을 반환
}
```

숫자1: 10
숫자2: 20



메소드 정의와 예제

return

메소드를 종료하는 기능

return 키워드가 실행 되면 현재 메소드는 종료되고 현재 메소드를 호출한 메소드의 실행 위치로 돌아감

반환해야 할 데이터가 있는 경우

반드시 return 키워드 뒤에 메소드의 선언 부에서 정의한 데이터 타입과 일치하는 데이터를 기술함

메소드 정의와 예제

메소드 예제2

```
package chapter01;

public class AddMethod {

	// 클래스를 실행하면 main() 메소드를 호출해서 실행합니다.
	public static void main(String[] args) {
		long result = add(100, 200);
		System.out.println(result);

		addPrint(10, 11);
	}
```

메소드 정의와 예제

메소드 예제2

```
// 선언 부
// 반환 데이터 타입은 long
// 메서드 이름은 add
// 매개변수 long n1, long n2로 정의
public static long add(long n1, long n2) {
        long result = n1 + n2; // 매개변수의 덧셈연산
        return result; // return 키워드는 값을 반환
}
```

메소드 정의와 예제

메소드 예제2

```
        // 반환 데이터타입 대신에 void 사용
        // 메소드 이름은 addPrint
        // 매개변수 long n1, long n2로 정의
        public static void addPrint(long n1, long n2) {
                long result = n1 + n2;
                System.out.println(result);
        }

}
```

결과(Console)

```
300
21
```